OntoDB: A B+ Tree Ontology Database with Hierarchy Intervals and Vector Search

Lam Nguyen
Microsoft
USA
lamng3.work@gmail.com

Abstract

An ontology store is asked three kinds of question: for the triples that match a pattern, for the instances at or below a class, and for the terms closest in meaning to a given term. We describe OntoDB, an open-source ontology database in C++20 built from the parts of a disk-based DBMS (dictionary-encoded triples in three B+ tree permutations, a buffer pool with LRU-K eviction, latch crabbing, a bulk loader, and page-level write-ahead logging) and extended to the last two questions. We give its algorithms for search, insert, delete, range scan, and bulk load, and a bound on the pages a scan reads. A postorder labelling of the class DAG with an interval set per class, kept in the engine's own B+ tree, turns "everything under C" into a few range scans: on the Gene Ontology the mean class needs 2.0 intervals, and the store holds 8.1× fewer triples than materializing the closure and reads 48× fewer pages than rewriting the query. We add exact, IVF (posting lists as B+ tree key ranges), product-quantized, and HNSW vector indexes, prove that no linear order of points keeps neighbors close, and show that in an ontology a class filter is correlated with the vectors: post-filtered IVF finds 7% of the right answers for a query outside a small class, and exact pre-filtering through the intervals is 10^2–10^4× faster than any index strategy below about 1 000 members. Against Oxigraph on LUBM(10), OntoDB is 1.6–4.0× faster on selective patterns, equal on large scans, and 7× slower to load; for the instances of a class on the Gene Ontology it is about 2× faster than Oxigraph on RocksDB and slower than its in-memory store once answers are decoded to strings. The engine also compiles to a 0.85 MB WebAssembly module that runs in a browser. Every number in this paper regenerates from scripts in the repository.

1 Introduction

Ontologies are stored and queried as sets of triples, and the engines that hold them have to answer three kinds of question. A *pattern* question ("which resources have predicate p and object o") is the base case, and sorted indexes over permutations of subject, predicate, and object answer it with one range scan [27, 35]. A *structural* question ("which instances have a type at or below class C") depends on the class hierarchy, which a plain index does not know. A *semantic* question ("which terms are closest in meaning to t") has no answer in the data at all and needs a similarity measure over embeddings. The three meet in "the terms most similar to t that lie under C".

Existing stores split these jobs. General RDF stores such as Apache Jena TDB2 [3] and Oxigraph [31] are built for the first, answer the second by rewriting a query or by materializing entailments (as RDFox does [25]), and leave the third to a separate vector database. Each of these is a large system to deploy next to a small ontology, and none runs in a web page next to the document that uses it.

This paper describes OntoDB, an open-source ontology database in C++20 that is built from the parts of a textbook disk-based DBMS and extended to all three questions. It makes five contributions.

(1) **A B+ tree triple store with its algorithms.** Three B+ tree permutations over dictionary-encoded triples, a buffer pool with LRU-K eviction, crabbing latches, a bulk loader, and page-level write-ahead logging (Sections 3–6). We give the algorithms for search, insert, delete, range scan, and bulk load, with the invariants they keep and the cost of a scan (Lemma 1).
(2) **Hierarchies as intervals.** A labelling of the class DAG that makes "everything under C" a few range scans of the engine's own B+ tree, with a bound on the number of intervals (Lemma 4) and no stored entailment (Section 10).
(3) **Vector indexes in the engine.** Exact, IVF, product-quantized, and HNSW search, with IVF posting lists stored as key ranges of the triple B+ tree (Section 11), and a proof that no sorted order can replace them (Lemma 2, Section 9).
(4) **Filtered search through the hierarchy.** A comparison of pre-, post-, and masked filtering for "nearest under C" that shows the filter is correlated with the vectors in an ontology (Section 12).
(5) **A build that runs in a browser.** The same engine compiled to a 0.85 MB WebAssembly module, with its SPARQL, plans, and semantic search (Section 13).

We compare the store with Oxigraph on LUBM(10), and the hierarchy index with Oxigraph's property-path evaluation on the Gene Ontology (Sections 7 and 10); every number regenerates from the scripts in the repository.

2 Background and Related Work

RDF stores. Triple stores keep triples in sorted permutations. Hexastore indexes all six [35]; RDF-3X keeps six compressed B+ trees and prunes joins with statistics [27]; QLever builds read-mostly permutation indexes in bulk [5]; HDT is a compressed read-only format [10]. Virtuoso stores quads in a relational engine [9]. Among the systems we could run side by side, Apache Jena TDB2 uses B+ trees [3], RDF4J's native store uses B-trees [8], and Oxigraph [31] stores its indexes in RocksDB, a log-structured merge tree. OntoDB follows the B+ tree line and keeps three permutations, which we show suffice (Section 7). The B+ tree [6, 7] and its concurrency and recovery machinery [12, 21, 24] are textbook; our contribution is the choice of parts, their measured behavior on ontologies, and what is built on top.

Table 1: Stores for ontologies, as documented (October 2026). We ran only Oxigraph and OntoDB.

System	Triples are kept in	Hierarchy by	In a browser
Oxigraph [31]	RocksDB (LSM)	query only	in-memory
Jena TDB2 [3]	B+ trees	reasoner layer	no
RDF4J native [8]	B-trees	materialized	no
RDF-3X [27]	6 compressed B+ trees	query only	no
RDFox [25]	in memory	materialized	no
OntoDB	3 B+ trees	intervals	full engine

Inference. RDFS entailment is usually either *rewritten* into the query or *materialized* at write time, with maintenance by counting or by delete-and-rederive [15]. Section 10 adds a third option that stores no entailed triple.

Labelling a hierarchy. Agrawal, Borgida, and Jagadish [1] store a compressed transitive closure by numbering a spanning forest in postorder and giving each node a set of intervals; Grust [13] numbers XML trees in pre- and postorder so axes become regions of a plane. We use the former, on class DAGs, inside a B+ tree store, and measure it against the other two ways to answer a type query.

Vector search. Approximate nearest neighbor search has three families that matter here: inverted files and product quantization [19], proximity graphs, of which HNSW [23] is the standard, and disk-oriented graphs such as DiskANN [18]. FAISS [20] packages the first two. Pan, Wang, and Li survey vector database systems and the hybrid queries (attribute plus vector) that are hard for them [32]. ACORN [33] makes HNSW traverse only the nodes that pass a predicate.

Graph and RDF systems with vectors. Neo4j offers an HNSW vector index queried from Cypher [26]; Amazon Neptune Analytics offers vector similarity over graph nodes [2]; Fluree provides inline similarity functions and HNSW indexes for RDF [11]; GraphDB has had similarity indexes queried from SPARQL since version 8 [30]. In the documentation we read, none turns the class hierarchy into range-scannable intervals, in the engine that holds the triples, to restrict a vector search.

Embeddings of hierarchies. Euclidean embeddings of names are what we use; hyperbolic embeddings [28] represent hierarchies more faithfully and would change the vectors, not the indexes.

Adaptive radix trees. The ART [22] orders keys by byte, so its range scan is lexical; DuckDB persists it [17]. We discuss it in Section 15.

Databases in the browser. WebAssembly [16] lets a C++ engine run in a page. Oxigraph ships a JavaScript build with an in-memory store. OntoDB compiles the whole engine, including the paged B+ trees, the buffer pool, and the semantic extension, by replacing the two parts that need threads or a native library (Section 13).

3 Architecture

OntoDB is a single-process engine in C++20 with the layers of a conventional disk-based DBMS (Figure 1). A SPARQL query is parsed, bound against the dictionary, planned as a left-deep tree, reordered by the optimizer, and run by volcano-style executors that pull rows of term ids. The executors read triples through a `TripleStore` interface. Two stores implement it: a memory store, and the *indexed store*, which is the one this paper is about. It keeps each triple in three B+ trees (SPO, POS, OSP), all in one page file, behind a buffer pool with LRU-K eviction [29]. Terms are dictionary-encoded by a dictionary that is itself built from the same parts: a term heap and two B+ trees. Updates are logged page by page for ARIES-style recovery [24]. A semantic extension (Sections 9–12) sits beside the store and reads it through the same interface.

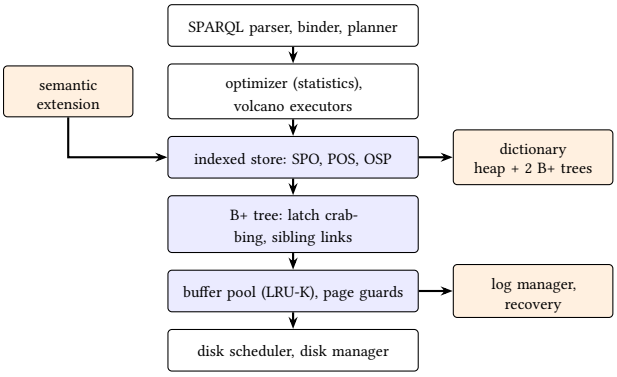


Figure 1: The layers of OntoDB. Blue boxes are what Section 4 describes.

Table 2: Which permutation answers which pattern (· is a free position). The bound positions are always a prefix.

Pattern	$s\,\cdot\,\cdot$	$\cdot\,p\,\cdot$	$\cdot\,\cdot\,o$	$\cdot\,\cdot\,\cdot$
Tree, prefix	SPO, 1	POS, 1	OSP, 1	SPO, 0
Pattern	$sp\cdot$	$\cdot po$	$s\cdot o$	spo
Tree, prefix	SPO, 2	POS, 2	OSP, 2	SPO, 3

4 Storage and Indexing

Keys and pages. A triple is a 24-byte key of three dictionary ids, stored big-endian, so `memcmp` is the order. A page is 4 KiB; a leaf holds up to B=169 keys and an internal page up to F=144 separators with F+1 child ids. Leaves are linked to their *right* sibling only. A header page names the root, so a root split changes one word and no tree changes identity. Every node but the root keeps at least half its maximum: a leaf at least $\lfloor B/2 \rfloor = 84$ keys, an internal page at least $\lfloor F/2 \rfloor + 1 = 73$ children.

Three permutations are enough. A triple pattern binds some of s, p, o. With SPO, POS, and OSP, every pattern has a permutation in which its bound positions form a prefix (Table 2), so a pattern is one range scan that returns only matching triples. Algorithm 1 picks the permutation with the longest bound prefix. The other three permutations (SOP, PSO, OPS) would make no pattern cheaper (Section 7).

Algorithm 1: ScanFor(s, p, o): choose a tree for a triple pattern

```
for each permutation π of SPO, POS, OSP do
    ℓ(π) ← the number of leading positions of π that the pattern binds;
π* ← the permutation with the largest ℓ (the first one on a tie);
k ← the bound values in the order of π*, padded with zeros;
return Begin(tree(π*), k, ℓ(π*)), keeping the triples that match the pattern;
```

Algorithm 2: LeafFor(k) and GetValue(k)

```
latch the header page (shared); r ← root; if r is nil then return nil;
g ← latch r (shared); release the header;
while g is an internal page do
    c ← ChildFor(g, k) ;          // binary search over the separators
    g′ ← latch c (shared); release g; g ← g′;
return g;
GetValue(k): g ← LeafFor(k); return whether k is in g (binary search);
```

4.1 Search

A point lookup and the start of every scan use the same descent. ChildFor picks the child whose range holds the key: the last separator that is $\le$ the key, or child 0 if none is. The descent *crabs*: it latches the child before it lets go of the parent, so a concurrent split cannot move the key away between two steps [12, 21].

A lookup reads h pages, where h is the height. Every non-root leaf holds at least 84 keys and every non-root internal page has at least 73 children, so a tree of n keys has at most $n/84$ leaves and height $h \le 2 + \log_{73}(n/84)$: at most four levels for the 1.3 million triples of LUBM(10), and three when the tree is bulk-loaded, as there it has about 7 800 full leaves under 54 internal pages.

4.2 Insert

Insert is *pessimistic* crabbing. It latches the header and then each page on the way down in write mode, and lets go of every latched ancestor, the header included, as soon as the page just latched is *safe*, meaning it cannot split. What is still latched at the leaf is exactly what a split can reach, so a split never needs a latch it does not already hold and the latch order (header, then top to bottom, siblings left to right) is the same everywhere.

A leaf split leaves two leaves at $\lfloor (B{+}1)/2 \rfloor$ and $\lceil (B{+}1)/2 \rceil$ keys, so the half-full invariant holds. Splits propagate only while the parent is full, so with random keys a split costs $O(1/B)$ page writes per insert on average. The only structural change that spans pages is a split or merge, and Section 6 makes that atomic for recovery.

4.3 Delete

Delete crabs the same way with a different notion of safe: a page that is above its minimum (for the root, more than one key), because only then can it not underflow. After removing the key from the leaf, a leaf below $\lfloor B/2 \rfloor$ keys *repairs* itself against a sibling under the same parent, preferring the left one.

A leaf emptied by a merge is *kept*, empty, with its sibling link still pointing on, rather than freed. An iterator may already hold its id (the next paragraph), and an empty page it can step through is cheaper than a protocol that tells every iterator the page is gone. Space reclaimed this way is not reused until the tree is rebuilt; freed

Algorithm 3: Insert(k)

```
latch the header (exclusive); if the tree is empty then make a leaf holding k the root; return true;
path ← [ ]; id ← root;
repeat
    g ← latch id (exclusive);
    if g is not full then
        release every latch in path and the header ; // g is safe
    push g on path; id ← ChildFor(g, k);
until the page is a leaf;
ℓ ← path.pop(); if k ∈ ℓ then return false;
insert k into ℓ; if |ℓ| ≤ B then return true;
// split the leaf: the left half stays, the right half moves, its first key is copied up
r ← new leaf with the upper half of ℓ; r.next ← ℓ.next; ℓ.next ← r;
sep ← r.keys[0];
while true do
    if path is empty then
        make a new root over (ℓ, r) with separator sep; set it in the header; return true
    q ← path.pop(); insert (sep, r) into q after ℓ;
    if q has at most F separators then return true;
    // split the internal page: the middle separator moves up and leaves both halves
    ℓ ← q; r ← new page with the upper half; sep ← the middle separator of q;
```

Algorithm 4: Remove(k)

```
descend as in Insert, releasing ancestors when the page just latched is safe (above its minimum);
ℓ ← the leaf; if k ∉ ℓ then return false;
remove k from ℓ; if ℓ is the root or |ℓ| ≥ ⌊B/2⌋ then return true;
while the node x on top of path is under its minimum and has a parent q do
    y ← the left sibling of x under q (the right one if x is first); latch siblings left to right;
    if |x| + |y| fits in one page then
        merge: move the right node's entries into the left (a leaf keeps its sibling link; an internal page pulls the separator down); delete the right entry from q;
    else
        redistribute: move one entry from y to x (a leaf copies a key; an internal page rotates through q) and fix the separator in q;
    x ← q;
if the root has one child left then make the child the root and free the old root;
```

Algorithm 5: BeginRange(lo, hi), Take, and Settle

```
ℓ ← LeafFor(lo); i ← the first position in ℓ with key ≥ lo;
Take(ℓ, i); Settle();
Take(ℓ, i): buf ← ⟨ℓ.keys[j] : j ≥ i, key < hi⟩; pos ← 0;
    next ← nil if some key ≥ hi was met, else ℓ.next; release ℓ (latch
    and pin);
Settle(): while pos ≥ |buf| do
    if next is nil then become the end iterator; return;
    Take(fetch(next), 0) ; // one sibling link, no climb to
      the root
Advance: pos ← pos + 1; Settle();
```

Algorithm 6: BulkLoad(K) for sorted, unique keys K

```
cut K into runs of B keys; if the last run has fewer than ⌊B/2⌋, it
  and the run before it share their keys evenly;
write each run as a leaf and link it to the previous one; L ← the list
  of (first key, page id);
while |L| > 1 do
    cut L into runs of F+1 children, again sharing the last two
      runs evenly if the last is too short;
    write each run as an internal page whose separators are the
      first keys of children 2..; L ← the (first key, id) of the new
      pages;
set the root in the header to the page of L[0];
```

internal pages are unreachable once their parent is updated and are released.

4.4 Range scan

The half-open range $[lo, hi)$ is the primitive the semantic extension needs (Section 10); a prefix scan is a range between PrefixLower and PrefixUpper. BeginRange descends to the leaf for lo, finds the first key $\geq lo$ by binary search, and hands the leaf to the iterator.

The iterator *copies* the keys of a leaf that lie below hi, remembers the right sibling's id, and releases the leaf. It holds no latch and no pin between rows, so a consumer that stalls (a join waiting on its other input, a client reading slowly) blocks no writer and pins no frame. Reading the next leaf is one page read and no descent. The cost is the copy and a weaker guarantee: between two leaves a writer may change the tree, so a scan sees each leaf as it was when it was read; the transaction layer's locks, not the latches, give a query its isolation. The iterator checks that keys increase along the chain and that a run of empty leaves does not cycle, and stops with an error otherwise.

Lemma 1 (Cost of a range scan). *A scan that returns m keys from an insert-only or bulk-loaded tree of height h reads at most $h + \lceil m/\lfloor B/2 \rfloor \rceil$ pages, and at most $h + \lceil m/B \rceil$ in a bulk-loaded tree.*

Proof. The descent ends at the first leaf, which is one of the h pages. Every leaf after it holds at least $\lfloor B/2 \rfloor$ keys, all inside the range except possibly the last, so at most $\lceil m/\lfloor B/2 \rfloor \rceil$ more leaves are read. A bulk-loaded leaf (below) is full, which replaces $\lfloor B/2 \rfloor$ by B. □

4.5 Bulk load

Loading a file inserts hundreds of thousands of triples into an empty store, and inserting one at a time leaves leaves about half full. OntoDB instead sorts the triples once for each permutation and builds the tree bottom-up.

Each page is written once, in order, and no page is split. A bulk-loaded tree is full: LUBM(1) takes 10.8 MiB for three permutations by bulk load against 16.2 MiB by insertion, and a full scan reads 599 pages against 1 131.

4.6 Buffer pool

Pages live in a fixed set of frames. A *page guard* pins a frame and takes its latch for the guard's lifetime, and the destructor releases both, so no path leaves a page pinned. A frame is evicted by LRU-K with K=2 [29], which ranks a frame by the time of its second most recent use and so is not flooded by one long scan. Reads and writes go through a disk scheduler; in the WebAssembly build (Section 13) the scheduler runs requests inline.

5 The Dictionary and the Query Pipeline

Dictionary. Terms (IRIs, literals with language or datatype, blank nodes) have one canonical spelling and get a dense 64-bit id in the order they first appear. The mapping is stored in pages: the spellings in a heap, $id \to record$ in a B+ tree keyed (id, $page$, $slot$), and $spelling \to id$ in a B+ tree keyed (hash, id, 0) where the hash is 64-bit FNV-1a. A lookup scans the hash prefix and compares the spellings of the candidates, so a collision costs a comparison and not a wrong answer. Ids are in first-seen order, not hierarchy order, a fact that Section 9 measures.

Planning and joins. The planner builds a left-deep tree of triple scans in textual order and the optimizer reorders it greedily: it appends, at each step, a pattern that shares a variable with what is already joined (any pattern, if none does), the one whose estimated join output is smallest; the output of a join is estimated as $|L|\,|R| / \max(d_L, d_R)$ for the distinct counts d of the join variable. The counts are exact and the store collects them at load, for every position, every pair of positions, and every triple (about seven hash entries per triple). The join operator is a nested loop, an index nested loop (the inner scan is reopened with the outer row's bindings as constants, so it is again a prefix scan of one tree), or a hash join; `auto` chooses per node. The SPARQL subset is basic graph patterns with `FILTER`, `DISTINCT`, `ORDER BY`, `LIMIT` and `OFFSET`, and `INSERT DATA` and `DELETE DATA`; `OPTIONAL`, `UNION`, property paths, and aggregates are not yet implemented, which Section 15 returns to.

6 Transactions and Recovery

Updates run under strict two-phase locking with a deadlock detector. Recovery is page-oriented: when a dirty page is released, the pool appends a record holding the page's image and stamps the page with its log sequence number; before a page is written, the pool forces the log to that stamp. Redo therefore copies page images whose stamp is older than the record and needs no knowledge of the B+ tree layout. Undo of a triple is *logical* and writes a compensation record, because other transactions may have changed

Table 3: Page reads per pattern on LUBM(1) (cold 64-frame pool, bulk-loaded) and file size, by set of permutations.

Permutations	$\cdot p \cdot$	$\cdot\cdot o$	$\cdot po$	file (MiB)
SPO	599	599	599	6.1
SPO, POS, OSP	76	15	15	10.8
all six	76	15	15	17.9

the page since. A split or merge is the one update that spans pages: INSERT and REMOVE open a *page group* that holds the log back from the group's first record until it ends, and the pool will not choose a page of the group as a victim, so a crash never leaves half a split on disk. A split or merge is redone and never logically undone. Commit forces the log, and checkpoints are sharp (flush every page, then write the record). Crash points sit between the pages of every multi-page operation, and a harness kills the process at each and checks the database after restart. The checks cover the invariants of Section 4: sorted keys, separators consistent with leaves, a leaf chain that visits every key once, and the half-full bound.

7 Evaluation of the Store

We compare OntoDB with Oxigraph [31], a Rust SPARQL 1.1 store whose default engine keeps triples in RocksDB, an LSM tree. It is the closest system we could run side by side: embeddable, open source, single-process, without entailment, and with a build that also runs in the browser. Section 2 lists the systems we did not run and why.

Setup. Both read the same files (the RDF/XML of the LUBM generator [14], 15 files for LUBM(1) and 189 for LUBM(10)), parse them with their own parser (raptor for OntoDB, Oxigraph's for the other), load them with their bulk loaders, and run the same query text in-process on one thread of an Apple M2 Pro with 16 GiB, a release build, and warm caches. OntoDB uses the indexed store with SPO, POS, and OSP and a 256 MiB buffer pool, which holds LUBM(10); Oxigraph 0.5.13 is used through its Rust API with RocksDB on disk and, separately, with its in-memory store. Each query is run once to warm and then 5 to 9 times; we report the median. Rows are converted to strings in both engines, and the row counts of the two agree on every query. The query is parsed once outside the timer in both; planning and evaluation are timed. The queries are the 14 LUBM queries and 100 single-pattern queries per shape, with constants sampled from the data, so that every pattern shape of Table 2 is measured. OntoDB reads 1 316 700 statements from LUBM(10), of which Oxigraph stores 1 272 953 distinct triples. The benchmark is `benchmarks/compare`; it regenerates every number here.

Permutations. One tree, SPO, answers a pattern that does not bind the subject by a scan of the whole tree. Table 3 counts, on LUBM(1) with a cold 64-frame pool, the page reads of a pattern for one, three, and all six permutations (averages over 20 sampled patterns per shape). Three permutations cut the pages of $\cdot p \cdot$ by 7.9× and those of $\cdot\cdot o$ and $\cdot po$ by 40×; the other three trees cost 66% more space and change no pattern, because every shape already has a prefix scan in the first three.

Table 4: Loading LUBM in each engine (bulk loader, one thread of the loader, files on a local SSD). Oxigraph's count is distinct triples; OntoDB's is statements read.

Data	Engine	triples	load (s)	disk (MiB)	RSS (MiB)
LUBM(1)	OntoDB	103 104	1.02	10.8	322
	Oxigraph, RocksDB	100 573	0.13	17.8	60
	Oxigraph, memory	100 573	0.18	—	53
LUBM(10)	OntoDB	1 316 700	13.83	133.7	637
	Oxigraph, RocksDB	1 272 953	1.95	176.3	87
	Oxigraph, memory	1 272 953	2.32	—	546

Table 5: Single-pattern queries on LUBM(10), 100 sampled constants per shape ($\cdot p \cdot$: every predicate): mean time per query (99th percentile in parentheses). Row counts agree between engines.

Pattern	rows	OntoDB	Oxigraph, RocksDB	Oxigraph, memory
$s\cdot\cdot$	7.3	17 µs (28 µs)	28 µs (39 µs)	12 µs (14 µs)
$sp\cdot$	1.6	4 µs (7 µs)	16 µs (20 µs)	10 µs (10 µs)
$\cdot\cdot o$	16,233.0	33.4 ms (162.3 ms)	33.6 ms (160.0 ms)	9.6 ms (47.3 ms)
$\cdot po$	16,225.2	17.3 ms (83.0 ms)	18.6 ms (91.8 ms)	7.1 ms (35.6 ms)
$\cdot p \cdot$	70,719.6	148.9 ms (460.8 ms)	146.4 ms (451.1 ms)	40.5 ms (128.5 ms)

Load and space. Table 4 gives the load. Oxigraph loads LUBM(10) 7.1× faster than OntoDB (1.95 against 13.83 s) and with a seventh of the memory. We did not profile Oxigraph. In OntoDB the time goes to parsing RDF/XML through raptor, interning every term through two B+ trees (on LUBM(1), the permutation study below attributes the time to the dictionary and not to the permutation trees), and building the exact statistics; the memory is the 256 MiB pool and those statistics. OntoDB's file is 24% smaller than the RocksDB directory (133.7 against 176.3 MiB): a bulk-loaded B+ tree is packed with no per-key overhead, and RocksDB holds a block index and filters, and compression was not tuned for either system.

Triple patterns. Table 5 gives the mean and 99th percentile over 100 sampled patterns per shape. For patterns that return few rows, OntoDB is faster than Oxigraph on RocksDB: 1.6× for $s\cdot\cdot$ and 4.0× for $sp\cdot$ on LUBM(10). Both are an index descent and a few keys; we did not profile Oxigraph to say where its extra time goes. For patterns that return thousands of rows the two are within a few percent of each other ($\cdot\cdot o$: 33.4 against 33.6 ms). Oxigraph's in-memory store is 3.5× faster than either on large results and within a factor of two of OntoDB on small ones. That is the comparison OntoDB's disk-oriented design does not win, and it is the cost of paging: a result is read out of pages through guards and copied, where a memory store follows pointers.

LUBM queries. Ten of the fourteen LUBM queries return no rows without inference; both engines answer them in microseconds and the ratios of two tiny times mean nothing, so Table 6 lists the four that return rows (all fourteen are in `benchmarks/compare/results.md`). On the triangle query (q2) OntoDB takes 8.0 ms against 253.6 ms on RocksDB and 30.5 ms in memory. OntoDB's plan starts from the most selective pattern and runs index nested-loop joins, each

Table 6: The LUBM queries that return rows without inference, on LUBM(10): median time.

Query	rows	OntoDB	Oxigraph, RocksDB	Oxigraph, memory
q01	4	12 μs	38 μs	16 μs
q02	28	8.0 ms	253.6 ms	30.5 ms
q03	6	17 μs	41 μs	17 μs
q14	75,547	84.8 ms	89.8 ms	33.3 ms

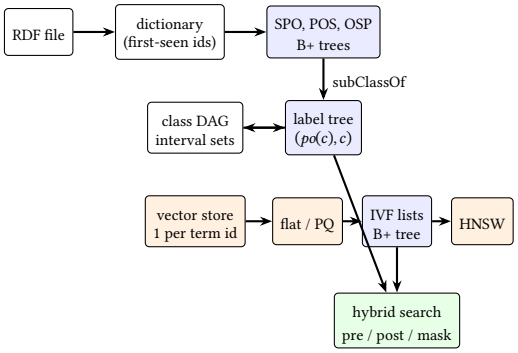


Figure 2: What OntoDB holds. Blue boxes are B+ trees of the same kind; the class DAG is rebuilt from the `subClassOf` triples, and vectors come from an external embedding model.

inner scan a prefix scan; we did not inspect Oxigraph's plan. On the large scan (q14) the two are equal.

What the comparison does not show. It does not show OntoDB faster in general. Oxigraph implements all of SPARQL 1.1 and OntoDB a subset; it loads much faster; it has a transaction story we did not test; and its in-memory store beats OntoDB on large results. What it shows is that a conventional B+ tree store with three permutations, a buffer pool, and an optimizer driven by exact counts is competitive with an LSM-based store on basic graph patterns, which is what the next sections need as a base.

8 The Semantic Extension

Figure 2 shows what the semantic extension adds to the store of Section 4. The triples are dictionary-encoded and indexed in three B+ tree permutations. The class hierarchy is read from the `rdfs:subClassOf` triples, labelled, and kept in a fourth B+ tree. Vectors, one per term id, sit in a store beside them, with three optional indexes; the IVF index is again a B+ tree. A hybrid query touches the label tree and one vector index, never the triple trees.

9 What a Sorted Index Can Say About Meaning

The order of OntoDB's trees. A triple is a 24-byte key of three dictionary ids compared with `memcmp`. The dictionary assigns ids densely, in first-seen order, and the loader interns the terms of each statement in file order. So the order of every tree is the order in which terms first appeared in the input. Two classes are adjacent in the index only if they were adjacent in the file.

Table 7: Share of the 10 nearest neighbors (by cosine) found within $\pm W$ positions of a term, and the median position gap to the nearest neighbor. GO, 38 092 terms, 2 000 queries.

Order	gap	±50	±100	±500	±1000	±5000
dictionary id (first seen)	1739	.195	.211	.258	.284	.438
hierarchy, deepest parent	45	.328	.377	.498	.561	.735
hierarchy, first parent	37	.319	.366	.481	.546	.726
label, alphabetical	1749	.310	.334	.379	.408	.558
random	10823	.003	.005	.024	.049	.242

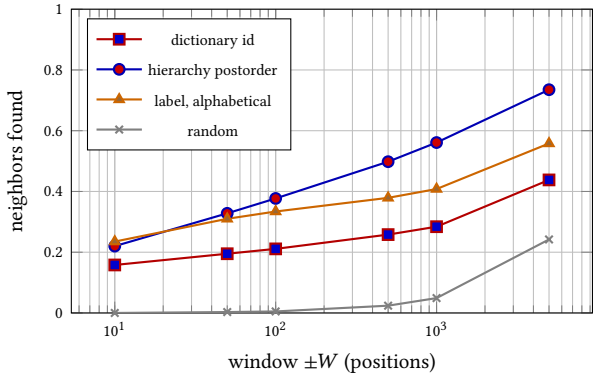


Figure 3: The data of Table 7: neighbors found in a window of $\pm W$ positions, by order.

No order holds similarity.

Lemma 2 (Bandwidth). *Let G have n vertices and diameter D. In every linear order of the vertices some edge joins vertices at least $(n-1)/D$ positions apart.*

Proof. The first and last vertices of the order are $n-1$ apart and joined by a path of at most D edges; some edge of the path covers at least $(n-1)/D$. □

For the neighbor graph of a d-dimensional grid with $n = k^d$ points (diameter $d(k-1)$) this gives $(k^d-1)/(d(k-1)) \geq k^{d-1}/d$ positions, in any order. A scan of width w cannot contain all neighbors unless w is that large. Space-filling curves keep neighbors close on average and need several scans to cover a query ball; they do not escape the lemma.

How much locality do real orders give? We took the Gene Ontology (release 2026-07-26, without obsolete terms: 38 092 classes, 57 456 `is_a` edges), embedded each term's name with `all-MiniLM-L6-v2` [34] (384 dimensions, on a CPU, 45 s), and for 2 000 random terms took the ten nearest by cosine. For each order of the terms, Table 7 gives the share of those neighbors within $\pm W$ positions.

Three things follow. First, the hierarchy's postorder is the best order, and it is much better than OntoDB's first-seen ids: the nearest neighbor is 45 positions away instead of 1 739. Second, even so, a scan of 100 positions finds 38% of the ten neighbors, and a window of ±1000 positions (2.6% of the ontology) still misses 44%; Lemma 2 is not an artifact of the grid. Third, first-seen ids are not

Algorithm 7: Interval sets for a class DAG

Input: edges (c, p): c is a subclass of p
Output: $po(c)$ and an interval set $I(c)$ for each class

collapse strongly connected components (Tarjan), number them parents first;
for *each component c, parents first* **do**
  $depth(c) \leftarrow 1 + \max_p\ depth(p)$; `// 0 for a root`
  $tp(c) \leftarrow$ the parent with the largest depth;
number the forest $\{(c, tp(c))\}$ in postorder; record $lo(c)$;
for *each component c, children first* **do**
  $I(c) \leftarrow \textsc{Merge}\big(\{[lo(c), po(c)]\} \cup \bigcup_{c \to v} I(v)\big)$; `// sort, join overlapping or adjacent`

random: the Gene Ontology's ids were minted in related batches, so ids carry some locality (0.21 at ±100) that depends on the file and not on the engine. Of the ten neighbors, 12% are a parent or child and 32% are within two `is_a` steps; a two-step neighborhood holds 45 terms (0.12% of the ontology), so hierarchy proximity is highly informative about meaning without being sufficient.

The order is therefore useful for structure and not a substitute for a vector index.

10 Hierarchies as Intervals

Labelling. Let the class hierarchy be a DAG, after collapsing each cycle ($A \sqsubseteq B \sqsubseteq A$) into one class, since such classes are equivalent. Choose a spanning forest by taking, for each class with several parents, the parent with the longest path from a root. Number the forest's nodes $1..n$ in postorder, and let $lo(c)$ be the smallest number in the tree subtree of c. For a forest, d is at or below c iff $lo(c) \le po(d) \le po(c)$. For a DAG each class carries a *set* of intervals, built bottom-up (Algorithm 7).

Lemma 3 (Correctness). *d is at or below c iff $po(d)$ lies in an interval of $I(c)$.*

Proof. Induction over the order in which Algorithm 7 builds I: the numbers covered by $I(c)$ are exactly those of the descendants of c. □

Lemma 4 (Size). *Let $E'(c)$ be the non-tree edges with tail at or below c. Then $|I(c)| \le 1 + |E'(c)|$.*

Proof. A descendant d is reached from c by a path. If it uses no non-tree edge, d is in the tree subtree of c; otherwise let v be the head of the last non-tree edge on it, and d is in the tree subtree of v. So the covered set is $\text{own}(c) \cup \bigcup_{e \in E'(c)} \text{own}(\text{head}(e))$, a union of $1 + |E'(c)|$ intervals. □

The labelling takes $O(n + m)$ plus the merges. A tree needs one interval per class. We check Lemma 4 on every class of the Gene Ontology and on 60 random DAGs in the test suite.

Index. The labels go into B+ trees of OntoDB: key $(po(c), c, 0)$ for the classes, and $(po(t), x, 0)$ for each triple $(x, type, t)$. The descendants of c, or the instances of c and its subclasses, are one range scan per interval of c. An instance insert adds one key. Labels are rebuilt when the hierarchy changes (here in 15 ms for GO); incremental maintenance with gaps [1] is not built. Dictionary ids are untouched: ids appear in three permutations, the log, and snapshots, and renumbering them to follow the hierarchy would rewrite all of it.

Table 8: Interval labels for the Gene Ontology.

Spanning-forest rule	non-tree edges	intervals	mean / p99 / max	build
deepest parent	19 367	76 570	2.01 / 17 / 741	14.9 ms
first parent	19 367	87 769	2.30 / 22 / 740	14.4 ms

Table 9: Descendants of a class in GO, cold 64-frame pool, mean over up to 40 classes per row.

Below C	method	scans	page reads	μs
10–99	intervals	5.9	5.8	59
	one scan per class	30.6	16.3	149
100–999	intervals	29.1	13.2	147
	one scan per class	268.2	65.7	803
1000+	intervals	101.3	40.1	528
	one scan per class	3303.6	694.1	8998

10.1 Evaluation

Setup. Gene Ontology as above; LUBM(1) [14] with its ontology (43 classes, 103 410 triples); and 200 000 synthetic entities, one to three types each drawn uniformly from GO's classes (495 552 triples), because GO has no instances. Release build on an Apple M2 Pro, one thread. Page reads are counted on a cold 64-frame buffer pool, the methodology of OntoDB's indexing benchmark.

Label size. Table 8 shows that GO needs 76 570 intervals for 38 092 classes, 2.01 per class, 598 KiB in all; the closure it replaces has 393 448 ancestor pairs, 9 MiB at 24 bytes each and one permutation. The spanning-forest rule matters little (deepest parent saves 13%). The tail is long: one class has 741 intervals, bounded by Lemma 4. Labels build in 15 ms. Asking whether d is below c takes 27 ns against 775 ns for a search up the parents.

Descendants of a class. Table 9 compares the B^+-tree interval scans with a breadth-first search that scans POS once per class. For a class with a thousand or more descendants the intervals need 101 scans instead of 3 304, 40 page reads instead of 694, and 0.5 ms instead of 9 ms.

Instances of a class. Table 10 compares the three strategies on the synthetic instances. Materializing the closure turns 495 552 triples into 4 017 973 (8.1×), the file from 63.6 to 309.5 MiB, and the load from 5.1 s to 180 s. The interval index adds 9.3 MiB (15%) and 0.17 s. On queries, the interval index is close to the materialized store and far from rewriting: for a class with a thousand or more descendants it reads 265 pages against 182 (materialized) and 12 803 (rewrite), and runs in 4.6 ms against 3.8 and 82 ms. All three return identical answers on every sampled class. On LUBM(1) the hierarchy is shallow (43 classes) and the three are within a factor of about two on

Table 10: Instances of C on 200 000 synthetic entities over GO, mean over 30 classes per row (cold 64-frame pool). Top: cost of the store.

Strategy	triples	file (MiB)	load (s)
rewrite	495 552	63.6	5.09
intervals	495 552	72.9	5.26
materialize	4 017 973	309.5	180.31

Below C	strategy	answers	page reads	μs
10–99	intervals	278	8.4	90
	rewrite	278	87.5	571
	materialized	278	6.6	444
100–999	intervals	2 954	40.9	1 461
	rewrite	2 954	1 153	7 280
	materialized	2 954	22.5	987
1000+	intervals	29 904	264.8	4 555
	rewrite	29 904	12 803	82 441
	materialized	29 904	182.0	3 776

Table 11: Instances of a class and its subclasses, 200 000 entities over the Gene Ontology, 30 classes per row, warm: mean time. OntoDB answers by interval scans; Oxigraph by `?x a ?c . ?c rdfs:subClassOf* <C>`. OntoDB "ids" stops before the dictionary; "strings" decodes every answer, as Oxigraph does.

Below C	answers	OntoDB ids	OntoDB strings	Ox., RocksDB	Ox., memory
1	10	1 μs	11 μs	37 μs	13 μs
2-9	40	3 μs	42 μs	91 μs	23 μs
10-99	278	10 μs	275 μs	539 μs	110 μs
100-999	2,954	127 μs	2.9 ms	5.4 ms	1.1 ms
1000+	29,904	1.7 ms	29.1 ms	60.6 ms	21.8 ms

every query; materialization adds 15% to the triples. The benefit of intervals grows with the depth and breadth of the hierarchy, and LUBM has little of either.

Against a property path. The strategies above are all OntoDB's own. To compare with another engine we ask Oxigraph, which stores no entailment, for the same instances over the same data with the query `?x a ?c . ?c rdfs:subClassOf* <C>`, on the same 30 classes per row with warm caches. The answer counts agree on every class. Table 11 gives OntoDB twice. The interval scan alone, which returns term ids, takes 1.7 ms for the 29 904 answers of the largest classes (58 ns each). Returning spellings, as Oxigraph does, adds the dictionary, and then OntoDB takes 29.1 ms: decoding is 94% of the time. At that level OntoDB is 1.9 to 3.3× faster than Oxigraph on RocksDB, and slower than its in-memory store by 1.3 to 2.5× for classes with two or more instances. The honest summary is narrower than the page counts above suggest: the intervals make the structural step cheap, and a vector filter uses them without decoding anything (Section 12), but an engine that must return spellings is limited by its dictionary.

11 Vector Indexes in the Engine

The engine keeps one unit-length float vector per term id, in memory, loaded from a checksummed file whose records name terms by spelling, so a file survives a dictionary that numbers terms differently. Embeddings come from outside; OntoDB stores and searches them. Four index families are available, each a different trade between exactness, memory, and build time.

Flat. Score every vector: $O(nd)$ and exact. With 8-bit scalar quantization (one scale per vector, $s = \max|x|/127$) each coordinate is off by at most $s/2$, so for a unit query a score is off by at most $(s/2)\sqrt{d}$, in a quarter of the memory.

IVF.. Spherical k-means with k-means++ seeding [4] gives L centroids; each vector goes to its nearest. A query scores the centroids and reads the p nearest lists. The cost is $Ld + p(n/L)d$, minimal at $L = \sqrt{pn}$. The design choice is where the lists live: *as key ranges of the B+ tree that holds the triples.* The pair (list, row) is a key (list, row, 0), so a list is a prefix scan, an insert is a tree insert, and the lists page, cache, and latch like every other index of the engine. Under a filter the search keeps probing the nearest remaining lists until it holds k allowed rows, so a filter that excludes everything near the query costs more instead of returning nothing.

Product quantization. Cut a vector into m pieces and replace each by the index of its nearest of 256 learned centers [19]: m bytes per vector. A query builds m tables of 256 dot products and scores a code as the sum of m lookups, which equals the dot product with the decoded vector, so all the error is on the stored side.

HNSW.. The layered proximity graph of Malkov and Yashunin [23], with the paper's neighbor-selection heuristic: a candidate is kept only if it is nearer the new node than to every neighbor already kept. Layer 0 allows $2M$ links. Search takes an optional filter and still crosses nodes it may not return.

11.1 Evaluation

38 092 GO terms, 384 dimensions, 1 000 query terms (the term itself excluded from its answer), exact search as ground truth, single thread. Table 12 gives selected points and Figure 4 the recall–latency curves.

A baseline that is not slow by accident. Our first run scored with a plain dot-product loop. A compiler may not vectorize it without `-ffast-math`, and the exact scan took 8.3 ms, which would have made the speed-ups look about twice as large: HNSW at ef=10 is 103 times faster than that baseline and 51 times faster than the vectorized one. With eight independent accumulators the exact scan takes 2.2 ms (13 Gflop/s), and every number here and in Section 12 uses that. We report it because a speed-up over "exact" depends entirely on how fast exact is.

What the numbers say. HNSW dominates on this data: recall 0.993 at 44 μs, 51× faster than the exact scan, for 3.9 MiB of links on top of the vectors. IVF needs 255 μs for 0.929 and 928 μs for 0.976. IVF over the B+ tree is about 2.7× more expensive per scored row than a contiguous scan (about 157 ns against 59 ns): each entry is a decoded 24-byte key and a random read of the vector. That is the price of lists that page, cache, and update with the triples. The cost

Table 12: Vector indexes on GO (recall@10, mean time per query, memory of the structure, build time).

Index	setting	recall	μs	MiB	build (s)
flat float32	exact	1.000	2 238	55.8	—
flat 8-bit	exact over codes	0.996	4 554	14.1	—
flat PQ m=48	codes only	0.778	819	1.7	10.4
flat PQ m=48	re-rank 100	0.998	905	1.7†	10.4
IVF L=512	nprobe 8	0.892	149	56.5	17.5
IVF L=512	nprobe 16	0.929	255	56.5	17.5
IVF L=512	nprobe 64	0.976	928	56.5	17.5
IVF L=512, 8-bit	nprobe 16	0.926	263	14.8	17.5
IVF L=512, PQ 96	nprobe 16	0.839	178	4.2	38.4
HNSW M=16	ef 10	0.993	44	59.7	15.0
HNSW M=16	ef 40	0.997	108	59.7	15.0
HNSW M=32	ef 160	0.999	474	63.4	28.0

† re-ranking reads the float vectors as well.

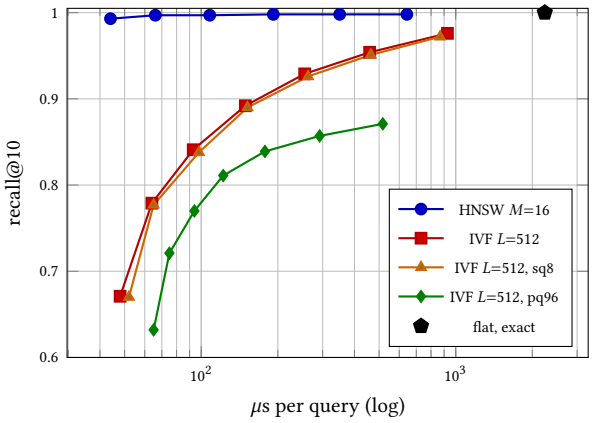


Figure 4: Recall@10 against time per query on GO (38 092 vectors, 384 dimensions).

model holds in shape: more lists make a query cheaper until the centroid scan dominates (L=2048 pays 124 μs at nprobe 1). Product quantization is for memory, not speed, at this size: codes alone reach 0.78 or 0.88 recall in 1.7 or 3.5 MiB, and re-ranking the best 100 exactly recovers 0.998 and 1.000 but needs the float vectors. 8-bit codes keep 0.996 recall in a quarter of the memory and are slower than float32, because converting bytes to floats costs more than the cache misses they avoid at 56 MiB.

Caveat. The queries are terms of the ontology, whose neighbors are also terms of it. Queries from outside the vocabulary, such as free text, would be harder, and we do not measure them.

12 Filtering a Vector Search by a Class

A hybrid query asks for the k terms nearest t that lie at or under class C. Let $s = |\text{subtree}(C)|/n$. We compare four ways of answering it, all returning terms of the subtree only.

Pre-filter. Resolve C to its members first (one range scan per interval), then score only those, exactly. About $s\,n\,d$ work.

Post-filter. Ask the index for $k' > k$, keep the results under C (a membership test is a binary search over C's intervals; nothing is scanned), and multiply k' by four until k remain or the index is exhausted. If the filter were independent of the similarity, about k/s candidates would be needed.

Masked search. Give the index a predicate, "the row's label lies in an interval of C", so a list scan or a graph walk skips rows outside C. Nothing is resolved in advance: each row the search touches costs one binary search. HNSW still crosses rows it may not return, to keep the graph connected, in the spirit of ACORN [33]; IVF keeps probing lists until it has k allowed rows.

Auto. Pre-filter when C has at most 4 096 classes, otherwise masked HNSW.

The filter is correlated with the vectors. In an ontology the names of a class's descendants are close to the class and to each other, so the neighbors of a term tend to lie under the term's own ancestors. Of the unfiltered top 100 neighbors of a query from *inside* C, 4% to 72% lie under C, rising with the size of C; for a query from *outside* C the share is 0% to 12%. The independence estimate k/s is therefore wrong in both directions: post-filtering is cheap for an inside query and does not terminate usefully for an outside one.

12.1 Evaluation

For 20 classes in each of five subtree-size buckets we ran the ten nearest terms under C for a query that is a member of C ("inside") and one that is not ("outside"), against the exact answer over the members. IVF uses 512 lists and 16 probes, HNSW M=16, ef=64. Table 13 gives recall and time.

Exact pre-filtering wins for small classes. Up to about 1 000 members, resolving the members through the interval index and scoring them exactly takes 3 to 62 μs and is the exact answer. Every index-based strategy is slower, from 1.7× (masked IVF at 101–1 000 members, at recall 0.96) to over 10 000× (post-filtered HNSW on the smallest classes), because few of the vectors an index touches belong to the class. A graph search for a rare filter crosses most of the graph: masked HNSW reads 16 ms for a class of at most 100 members, the failure ACORN is designed to remove.

Post-filtering fails for outside queries. IVF post-filter recall falls to 0.07 on the smallest classes and 0.64 on the largest; HNSW reaches full recall only by searching again and again (37 ms for a class of at most ten members, in six rounds).

Masked HNSW wins for large classes. Above 5 000 members it takes 105 μs inside and 900 μs outside, exact, against 1.4 ms for resolving and scoring the members.

Do not resolve the filter. Our first masked search resolved C by interval scans and built a bitmap per query. For a class of over 5 000 members that took 669 μs, of which about 560 μs was resolving. Testing the label of each row the search touches, one binary search over a handful of intervals, brought the same search to 105 μs. This is where interval labels earn their keep a second time: they make membership an $O(\log |I(C)|)$ test with no scan.

Table 13: Ten nearest terms under a class C, by subtree size, query inside or outside C: recall@10 / μs per query. GO, 38 092 terms. Masked and post-filtered searches use the index; pre-filter resolves the members and scores them exactly.

Strategy	≤10	11–100	101–1000	1001–5000	>5000
query inside C					
pre-filter	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 292	1.00 / 1 404
post-filter, HNSW	1.00 / 37 553	1.00 / 1 739	1.00 / 427	1.00 / 70	1.00 / 68
post-filter, IVF	0.86 / 903	0.87 / 558	0.93 / 293	0.98 / 198	0.99 / 211
masked HNSW	1.00 / 16 653	1.00 / 15 846	1.00 / 1 150	1.00 / 183	1.00 / 105
masked IVF	1.00 / 1 253	0.95 / 111	0.96 / 104	0.98 / 124	0.99 / 161
auto	1.00 / 3	1.00 / 9	1.00 / 59	1.00 / 269	1.00 / 95
query outside C					
pre-filter	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 291	1.00 / 1 394
post-filter, HNSW	1.00 / 36 654	1.00 / 27 669	0.99 / 13 607	1.00 / 5 513	1.00 / 836
post-filter, IVF	0.07 / 966	0.02 / 1 009	0.17 / 902	0.33 / 811	0.64 / 591
masked HNSW	1.00 / 16 802	1.00 / 16 611	1.00 / 8 697	1.00 / 4 424	1.00 / 900
masked IVF	1.00 / 1 255	0.79 / 698	0.45 / 286	0.37 / 147	0.66 / 107
auto	1.00 / 3	1.00 / 9	1.00 / 62	1.00 / 295	1.00 / 860

A cost model for the crossover. Let c_m be the cost of pre-filtering per member (an interval scan, a hash lookup from term to row, and one dot product): from Table 13, $c_m \approx 0.13$–0.20 μs for classes of a few hundred members or more. Pre-filtering beats a masked graph search when $|C| < T = c_h/c_m$, where c_h is the masked search's time. For a query inside C, $c_h \approx 105$–183 μs, so $T \approx 800$–$1\,000$, which matches the observed crossing between the 101–1 000 and 1 001–5 000 columns. For a query outside C the masked walk costs 0.9–4.4 ms on classes of 1 000 or more members, so T is 7 000–25 000. A single threshold has to sit between the two; 4 096 stays below the outside-query crossing and loses at most a factor of 3.8 at the inside-query one.

The auto rule. With a threshold of 4 096 members, auto matches the fastest strategy in every cell but one and is never slower than 0.9 ms. The exception is a class of 1 001–5 000 members with an inside query: 269 μs, against 70 μs for post-filtered HNSW and 183 μs for masked HNSW. Post-filtering would cost 5.5 ms for an outside query on the same class, so the rule gives up a factor of 3.8 in one cell to avoid a factor of 79 in its neighbor.

13 A Build for the Browser

The engine compiles with Emscripten to a 0.85 MB WebAssembly module [16] that runs the same indexed store, planner, optimizer, and semantic extension in a web page. Two parts of the native build do not carry over, and replacing them is the whole port.

No raptor. The native loader parses RDF/XML, Turtle, and N-Triples with raptor, a C library. The web build uses a 480-line Turtle and N-Triples parser of our own (prefixes, `a`, `;` and `,`, blank-node property lists, collections, long strings, escapes, language tags, datatypes, and numeric and boolean literals); a test checks that it yields the same triples as raptor on the shipped data. RDF/XML needs the native build.

No threads. The native engine has a disk-scheduler thread, a deadlock detector, and a worker per session. In the module the scheduler executes each request inline, the detector is not started, and a statement runs on the calling thread. There is one session, so there is nothing to deadlock. The buffer pool, the page file (held in the browser's memory file system), the B+ trees, and the plans are unchanged, so Table 2's page counts and the plans printed by `\explain analyze` are the native ones.

Using it. The module exports three functions (execute a statement, run a query and return JSON, reset), and the playground page on the project's site calls them. It offers the memory store and the B+ tree store on a buffer pool, shows the buffer-pool hits and misses of each query, and loads a Gene Ontology slice of 834 classes with 384-dimensional embeddings for the semantic commands. In a desktop browser, building the interval labels takes 3 ms, `\under "cell death"` (97 classes, one range scan) 0.3 ms, `\similar` with a class filter 0.8 ms, and building the IVF and HNSW indexes 58 and 98 ms. Updates are saved as a journal in the page's local storage and replayed when the page loads, which keeps a session across reloads but is not durability: there is no write-ahead log or recovery in the web build.

No GPU.. We considered WebGPU for the dot products and did not use it. A similarity query scores a few hundred vectors with HNSW or at most a few thousand exactly; in the module that is under a millisecond with 128-bit SIMD, which the build enables, and moving the query to a GPU and back costs more than that.

14 Implementation and Reproducibility

OntoDB is about 13 000 lines of C++20 for the store, the query pipeline, transactions, and recovery; 2 600 for the semantic extension; 900 for the shell; and 6 200 for tests, including property tests that compare every strategy with a naive oracle on random ontologies and a crash harness that kills the process at each multi-page step of an update. The semantic extension adds one B+ tree entry point to the store (a range scan between two keys). In the shell:

```
\load go.nt          \hindex build
\vload go.ovec       \vindex build hnsw
\similar "apoptotic process" 5 under "cell death"
 0.865 GO:1902742 apoptotic process involved
```

```
      in development            [pre-filter]
 0.802 GO:0060561 apoptotic process involved
      in morphogenesis
\under "programmed cell death" 5
96 classes (4 range scans, 0.012 ms)
```

Vectors are held in memory and the vector indexes are rebuilt, not updated, when the vectors change; labels are rebuilt when the hierarchy changes (15 ms for GO). The semantic engine owns a private buffer pool on a temporary file for the label tree and the IVF lists, so it runs on either of the store's backends.

Reproducing the numbers. Data and models are fetched, not shipped: the Gene Ontology `go-basic.obo` release 2026-07-26, embeddings by `all-MiniLM-L6-v2` through `sentence-transformers` on the CPU (45 s), and LUBM generated by the official generator. `benchmarks/semantic/run.sh` writes every table of Sections 9–12, and `benchmarks/compare/run.sh` the comparison with Oxigraph 0.5.13 (Rust API, RocksDB and in-memory stores), which `paper/make-compare.py` turns into the tables of Sections 7 and 10. Seeds are fixed, queries are the same for every method, and the machine is an Apple M2 Pro with 16 GiB, one thread, `-O3`. Run-to-run variation in times is a few percent; page counts, recalls, and row counts are deterministic.

15 Discussion

Is an ART better? An adaptive radix tree orders keys by byte [22], so its range scan is a lexical range scan, the same one-dimensional order as a B+ tree, and Lemma 2 applies. Giving its leaves sibling links is possible and awkward. Path compression and node growth never move a leaf, but lazy expansion does: inserting into a leaf turns it into an inner node and puts a *copy* of the leaf under it, so a link that pointed at the old object points at an inner node; each insert must also find its sorted neighbors by a partial descent, and concurrent inserts must latch both. The usual alternative is a stack-based iterator, with no links, that climbs and descends between leaves. OntoDB's keys are three fixed-width integers, which leave an ART little to compress, so the B+ tree stays.

Pessimistic or optimistic prefixes. A compressed prefix can be stored in the node (pessimistic) or only its length (optimistic, with the full key checked once at the leaf). Seeking a lower bound needs to know whether a whole subtree is below or above it. With stored bytes the seek decides at the node; with skipped bytes it must load a key from the subtree and compare. The upper bound needs no jumping: a second seek finds the first key past it, and each step compares two keys.

What OntoDB does not do. The SPARQL subset has no `OPTIONAL`, `UNION`, property paths, or aggregates, and there is no reasoner beyond `rdfs:subClassOf`. Oxigraph implements SPARQL 1.1 and Jena and RDF4J come with reasoners, so a user who needs the full language needs one of them; our comparison is on basic graph patterns because that is where the two overlap. The dictionary is the slowest part: decoding an id to its spelling costs about 1 μs (two B+ tree lookups under one mutex), which is 17 times the cost of the interval scan that produced the id (Table 11); a cache of decoded terms is the obvious next step and would change the string columns, not the id ones. Loading is single-threaded and 7 times slower than Oxigraph's. Peak memory during a load is high because the optimizer's exact statistics are built in hash tables.

Limits. The vectors in our measurements come from one ontology and one small model that embeds names only; definitions, synonyms, or a domain model would change the vectors, not the indexes. The instance data is synthetic, because GO has none, and LUBM's hierarchy is too shallow to separate the strategies. The semantic benchmarks are single-threaded and keep the vectors in memory. Labels are rebuilt, not maintained under edits to the hierarchy, and only `rdfs:subClassOf` is read; no reasoner. Embeddings are Euclidean; hyperbolic embeddings [28] would represent a hierarchy more faithfully and would change what the vectors say, not how they are searched. Query text needs an embedding model that OntoDB does not link; the shell searches from the vector of a term that is already stored.

What we did not find. We did not find a vector index that makes the hierarchy order unnecessary: neighbors by meaning and descendants are different sets (a third of a term's ten nearest neighbors are within two `is_a` steps, two thirds are not), which is why the two are combined at query time and not at index time.

16 Conclusion

An ontology store is asked for patterns, for hierarchies, and for meaning. A B+ tree store with three permutations, a buffer pool, and an optimizer driven by exact counts answers patterns as fast as an LSM-based store and faster when they are selective, and its algorithms (crabbing insert and delete, a prefix range scan that holds no latch between rows, a bulk loader that fills pages) are small enough to state in a page. Labelling the class DAG with interval sets turns "everything under C" into a few range scans of the same tree, at 2.0 intervals per class on the Gene Ontology and a fraction of the cost of materializing the closure, and makes membership in a class's subtree a binary search, which a filtered vector search needs. A sorted index orders; it does not say what is close in meaning, and the bandwidth argument and the Gene Ontology measurements agree. Exact pre-filtering through the intervals is the right answer for classes up to about a thousand members, and a masked graph search for the largest. All of it fits in about sixteen thousand lines, runs in a browser from a 0.85 MB module, and regenerates its numbers from one script per question.

References

[1] Rakesh Agrawal, Alexander Borgida, and H. V. Jagadish. 1989. Efficient Management of Transitive Relationships in Large Data and Knowledge Bases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 253–262. https://doi.org/10.1145/67544.66950

[2] Amazon Web Services. 2023. Amazon Neptune Analytics: Vector Similarity Search. AWS documentation and launch announcement. https://aws.amazon.com/blogs/aws/introducing-amazon-neptune-analytics-a-high-performance-graph-analytics, accessed October 2026.

[3] Apache Jena. [n.d.]. TDB2: a native triple store for Apache Jena. https://jena.apache.org/documentation/tdb2/. Accessed October 2026.

[4] David Arthur and Sergei Vassilvitskii. 2007. k-means++: The Advantages of Careful Seeding. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1027–1035.

[5] Hannah Bast and Björn Buchhold. 2017. QLever: A Query Engine for Efficient SPARQL+Text Search. In *Proceedings of the ACM Conference on Information and Knowledge Management*. 647–656.

[6] Rudolf Bayer and Edward M. McCreight. 1972. Organization and Maintenance of Large Ordered Indexes. *Acta Informatica* 1, 3 (1972), 173–189.

[7] Douglas Comer. 1979. The Ubiquitous B-Tree. *Comput. Surveys* 11, 2 (1979), 121–137.

[8] Eclipse RDF4J. [n.d.]. Eclipse RDF4J: a Java framework for RDF. https://rdf4j.org/. Accessed October 2026.

[9] Orri Erling and Ivan Mikhailov. 2010. Virtuoso: RDF Support in a Native RDBMS. In *Semantic Web Information Management*. Springer, 501–519.

[10] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. 2013. Binary RDF Representation for Publication and Exchange (HDT). *Journal of Web Semantics* 19 (2013), 22–41.

[11] Fluree PBC. 2026. Fluree Vector Search. Fluree documentation. https://labs.fluree.com/docs/db/indexing-and-search/vector-search, accessed October 2026.

[12] Goetz Graefe. 2011. Modern B-Tree Techniques. *Foundations and Trends in Databases* 3, 4 (2011), 203–402.

[13] Torsten Grust. 2002. Accelerating XPath Location Steps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 109–120.

[14] Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. 2005. LUBM: A Benchmark for OWL Knowledge Base Systems. *Journal of Web Semantics* 3, 2–3 (2005), 158–182.

[15] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining Views Incrementally. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*.

[16] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. 185–200.

[17] Pedro Holanda. 2022. Persistent Storage of Adaptive Radix Trees (ART) in DuckDB. DuckDB blog. https://duckdb.org/2022/07/27/art-storage.

[18] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[19] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[20] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[21] Philip L. Lehman and S. Bing Yao. 1981. Efficient Locking for Concurrent Operations on B-Trees. *ACM Transactions on Database Systems* 6, 4 (1981), 650–670.

[22] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 38–49.

[23] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[24] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems* 17, 1 (1992), 94–162.

[25] Boris Motik, Yavor Nenov, Robert Piro, Ian Horrocks, and Dan Olteanu. 2014. Parallel Materialisation of Datalog Programs in Centralised, Main-Memory RDF Systems. In *Proceedings of AAAI*.

[26] Neo4j, Inc. 2026. Neo4j Vector Index and Search. Neo4j documentation. https://neo4j.com/developer/genai-ecosystem/vector-search/, accessed October 2026.

[27] Thomas Neumann and Gerhard Weikum. 2010. The RDF-3X Engine for Scalable Management of RDF Data. *The VLDB Journal* 19, 1 (2010), 91–113.

[28] Maximilian Nickel and Douwe Kiela. 2017. Poincaré Embeddings for Learning Hierarchical Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[29] Elizabeth J. O'Neil, Patrick E. O'Neil, and Gerhard Weikum. 1993. The LRU-K Page Replacement Algorithm for Database Disk Buffering. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 297–306.

[30] Ontotext. 2019. GraphDB 8.9 with Improved Semantic Similarity Search. Ontotext announcement. https://sirma.com/news/graphdb-8-9-with-improved-semantic-similarity-search.html.

[31] Oxigraph contributors. [n.d.]. Oxigraph: a SPARQL graph database written in Rust. https://github.com/oxigraph/oxigraph. Version 0.5; accessed October 2026.

[32] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of Vector Database Management Systems. *The VLDB Journal* 33 (2024), 1591–1615.

[33] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 120.

[34] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.

[35] Cathrin Weiss, Panagiotis Karras, and Abraham Bernstein. 2008. Hexastore: Sextuple Indexing for Semantic Web Data Management. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1008–1019.